

Bignum

高精度计算

太长不看版：结尾自取模板.....

定义

高精度计算 (Arbitrary-Precision Arithmetic)，也被称作大整数 (bignum) 计算，运用了一些算法结构来支持更大整数间的运算（数字大小超过语言内建整型）。

引入

高精度问题包含很多小的细节，实现上也有很多讲究。

所以今天就来一起实现一个简单的计算器吧。

▼ 任务

输入：一个形如 `a <Top> b` 的表达式。

- `a`、`b` 分别是长度不超过 1000 的十进制非负整数；
- `<Top>` 是一个字符（`+`、`-`、`*` 或 `/`），表示运算。
- 整数与运算符之间由一个空格分隔。

输出：运算结果。

- 对于 `+`、`-`、`*` 运算，输出一行表示结果；
- 对于 `/` 运算，输出两行分别表示商和余数。
- 保证结果均为非负整数。

存储

在平常的实现中，高精度数字利用字符串表示，每一个字符表示数字的一个十进制位。因此可以说，高精度数值计算实际上是一种特别的字符串处理。

读入字符串时，数字最高位在字符串首（下标小的位置）。但是习惯上，下标最小的位置存放的是数字的 **最低位**，即存储反转的字符串。这么做的原因在于，数字的长度可能发生变化，但我们希望同样权值位始终保持对齐（例如，希望所有的个位都在下标 `[0]`，所有的十位都在下标 `[1]`.....）；同时，加、减、乘的运算一般都从个位开始进行（回想小学的竖式运算），这都给了「反转存储」以充分的理由。

此后我们将一直沿用这一约定。定义一个常数 `LEN = 1004` 表示程序所容纳的最大长度。

由此不难写出读入高精度数字的代码：

```

1 void clear(int a[]) {
2     for (int i = 0; i < LEN; ++i) a[i] = 0;
3 }
4
5 void read(int a[]) {
6     static char s[LEN + 1];
7     scanf("%s", s);
8
9     clear(a);
10
11     int len = strlen(s);
12     // 如上所述, 反转
13     for (int i = 0; i < len; ++i) a[len - i - 1] = s[i] - '0';
14     // s[i] - '0' 就是 s[i] 所表示的数码
15     // 有些同学可能更习惯用 ord(s[i]) - ord('0') 的方式理解
16 }

```

输出也按照存储的逆序输出。由于不希望输出前导零，故这里从最高位开始向下寻找第一个非零位，从此处开始输出；终止条件 $i \geq 1$ 而不是 $i \geq 0$ 是因为当整个数字等于 0 时仍希望输出一个字符 0。

```

1 void print(int a[]) {
2     int i;
3     for (i = LEN - 1; i >= 1; --i)
4         if (a[i] != 0) break;
5     for (; i >= 0; --i) putchar(a[i] + '0');
6     putchar('\n');
7 }

```

拼起来就是一个完整的复读机程序咯。

► copycat.cpp

四则运算

四则运算中难度也各不相同。最简单的是高精度加减法，其次是高精度—单精度（普通的 `int`）乘法和高精度—高精度乘法，最后是高精度—高精度除法。

我们将按这个顺序分别实现所有要求的功能。

加法

高精度加法，其实就是竖式加法啦。

$$\begin{array}{r}
 962 \\
 + \quad 93 \\
 \hline
 1055
 \end{array}$$

也就是从最低位开始，将两个加数对应位置上的数码相加，并判断是否达到或超过 10。如果达到，那么处理进位：将更高一位的结果上增加 1，当前位的结果减少 10。

```

1 void add(int a[], int b[], int c[]) {
2     clear(c);
3
4     // 高精度实现中，一般令数组的最大长度 LEN 比可能的输入大一些
5     // 然后略去末尾的几次循环，这样一来可以省去不少边界情况的处理
6     // 因为实际输入不会超过 1000 位，故在此循环到 LEN - 1 = 1003 已经足够
7     for (int i = 0; i < LEN - 1; ++i) {
8         // 将相应位上的数码相加
9         c[i] += a[i] + b[i];
10        if (c[i] >= 10) {
11            // 进位
12            c[i + 1] += 1;
13            c[i] -= 10;
14        }
15    }
16 }

```

试着和上一部分结合，可以得到一个加法计算器。

► adder.cpp

减法

高精度减法，也就是竖式减法啦。

$$\begin{array}{r}
 123 \\
 - 56 \\
 \hline
 67
 \end{array}$$

从个位起逐位相减，遇到负的情况则向上一位借。整体思路与加法完全一致。

```

1 void sub(int a[], int b[], int c[]) {
2     clear(c);
3
4     for (int i = 0; i < LEN - 1; ++i) {
5         // 逐位相减
6         c[i] += a[i] - b[i];
7         if (c[i] < 0) {
8             // 借位
9             c[i + 1] -= 1;
10            c[i] += 10;
11        }
12    }
13 }

```

将上一个程序中的 `add()` 替换成 `sub()`，就有了一个减法计算器。

► subtractor.cpp

试一试，输入 1 2——输出 /9999999，诶这个 **OI Wiki** 怎么给了我一份假的代码啊……

事实上，上面的代码只能处理减数 大于等于被减数 的情况。处理被减数比减数小，即 时的情况很简单。

要计算 的值，因为有 ，可以调用以上代码中的 `sub` 函数，写法为 `sub(b, a, c)`。要得到 的值，在得数前加上负号即可。

乘法

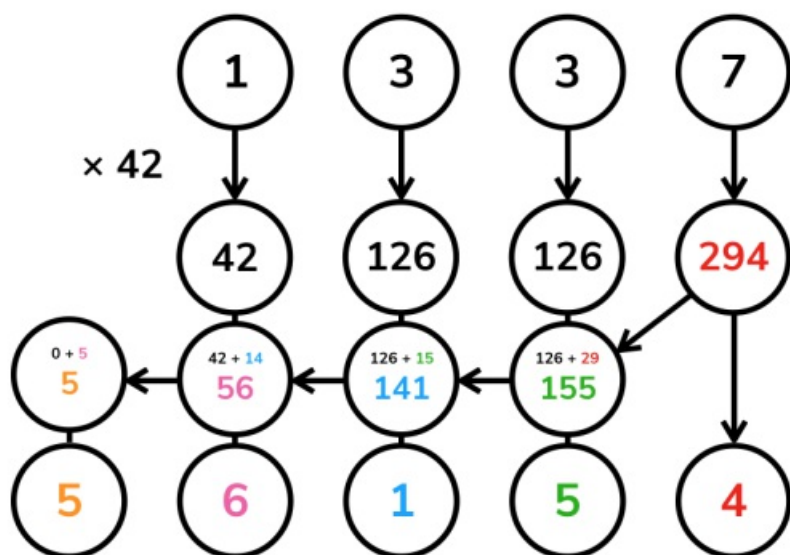
高精度—单精度

高精度乘法，也就是竖.....等会儿等会儿！

先考虑一个简单的情况：乘数中的一个普通的 `int` 类型。有没有简单的处理方法呢？

一个直观的思路是直接将 每一位上的数字乘以 。从数值上来说，这个方法是正确的，但它并不符合十进制表示法，因此需要将它重新整理成正常的样子。

重整的方式，也是从个位开始逐位向上处理进位。但是这里的进位可能非常大，甚至远大于 ，因为每一位被乘上之后都可能达到 的数量级。所以这里的进位不能再简单地进行 运算，而是要通过除以 的商以及余数计算。详见代码注释，也可以参考下图展示的一个计算高精度数 乘以单精度数 的过程。



当然，也是出于这个原因，这个方法需要特别关注乘数 的范围。若它和 （或相应整型的取值上界）属于同一数量级，那么需要慎用高精度—单精度乘法。

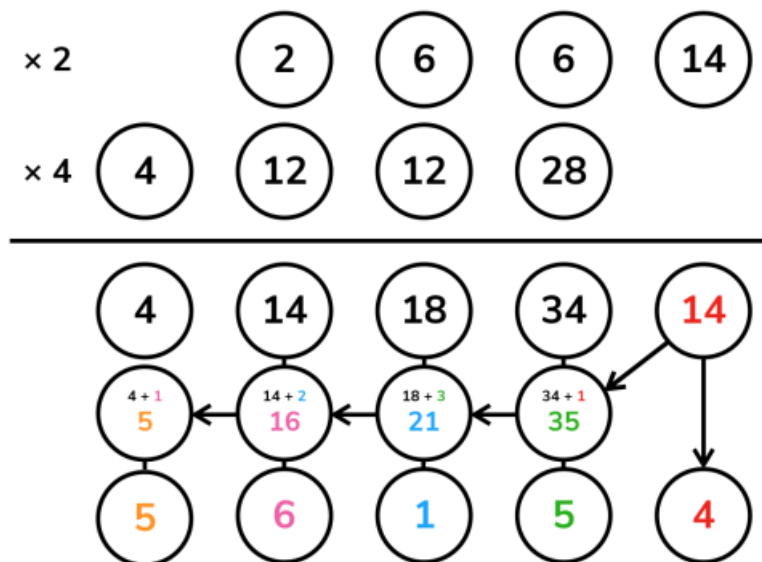
```
1 void mul_short(int a[], int b, int c[]) {
2     clear(c);
3
4     for (int i = 0; i < LEN - 1; ++i) {
5         // 直接把 a 的第 i 位数码乘以乘数，加入结果
6         c[i] += a[i] * b;
7
8         if (c[i] >= 10) {
9             // 处理进位
10            // c[i] / 10 即除法的商数成为进位的增量值
11            c[i + 1] += c[i] / 10;
12            // 而 c[i] % 10 即除法的余数成为在当前位留下的值
13            c[i] %= 10;
14        }
15    }
16 }
```

高精度—高精度

如果两个乘数都是高精度，那么竖式乘法又可以大显身手了。

回想竖式乘法的每一步，实际上是计算了若干 的和。例如计算 266×4 ，计算的就是 4×2 ， 4×6 ， 4×6 ， 4×14 。

于是可以将 分解为它的所有数码，其中每个数码都是单精度数，将它们分别与 相乘，再向左移动到各自的位置上相加即得答案。当然，最后也需要用与上例相同的方式处理进位。



注意这个过程与竖式乘法不尽相同，我们的算法在每一步乘的过程中并不进位，而是将所有的结果保留在对应的位置上，到后再再统一处理进位，但这不会影响结果。

```
1 void mul(int a[], int b[], int c[]) {
2     clear(c);
3
4     for (int i = 0; i < LEN - 1; ++i) {
5         // 这里直接计算结果中的从低到高第 i 位，且一并处理了进位
6         // 第 i 次循环为 c[i] 加上了所有满足 p + q = i 的 a[p] 与 b[q] 的乘积之和
7         // 这样做的效果和直接进行上图的运算最后求和是一样的，只是更加简短的一种实现方式
8         for (int j = 0; j <= i; ++j) c[i] += a[j] * b[i - j];
9
10        if (c[i] >= 10) {
11            c[i + 1] += c[i] / 10;
12            c[i] %= 10;
13        }
14    }
15 }
```

除法

高精度除法的一种实现方式就是竖式长除法。

$$\begin{array}{r}
 38 \\
 12 \overline{) 456} \\
 \underline{36} \\
 96 \\
 \underline{96} \\
 0
 \end{array}$$

竖式长除法实际上可以看作一个逐次减法的过程。例如上图中商数十位的计算可以这样理解：将 36 减去三次 12 后变得小于 12，不能再减，故此位为 3。

为了减少冗余运算，我们提前得到被除数的长度 与除数的长度，从下标 0 开始，从高位到低位来计算商。这和手工计算时将第一次乘法的最高位与被除数最高位对齐的做法是一样的。

参考程序实现了一个函数 `greater_eq()` 用于判断被除数以下标 `last_dg` 为最低位，是否可以再减去除数而保持非负。此后对于商的每一位，不断调用 `greater_eq()`，并在成立的时候用高精度减法从余数中减去除数，也即模拟了竖式除法的过程。

```

1 // 被除数 a 以下标 last_dg 为最低位，是否可以再减去除数 b 而保持非负
2 // len 是除数 b 的长度，避免反复计算
3 bool greater_eq(int a[], int b[], int last_dg, int len) {
4     // 有可能被除数剩余的部分比除数长，这个情况下最多多出 1 位，故如此判断即可
5     if (a[last_dg + len] != 0) return true;
6     // 从高位到低位，逐位比较
7     for (int i = len - 1; i >= 0; --i) {
8         if (a[last_dg + i] > b[i]) return true;
9         if (a[last_dg + i] < b[i]) return false;
10    }
11    // 相等的情形下也是可行的
12    return true;
13 }
14
15 void div(int a[], int b[], int c[], int d[]) {
16     clear(c);
17     clear(d);
18
19     int la, lb;
20     for (la = LEN - 1; la > 0; --la)
21         if (a[la - 1] != 0) break;
22     for (lb = LEN - 1; lb > 0; --lb)
23         if (b[lb - 1] != 0) break;
24     if (lb == 0) { // 除数不能为零
25         puts("> <");
26         return;
27     }
28
29     // c 是商
30     // d 是被除数的剩余部分，算法结束后自然成为余数
31     for (int i = 0; i < la; ++i) d[i] = a[i];
32     for (int i = la - lb; i >= 0; --i) {
33         // 计算商的第 i 位
34         while (greater_eq(d, b, i, lb)) {
35             // 若可以减，则减
36             // 这一段是一个高精度减法
37             for (int j = 0; j < lb; ++j) {
38                 d[i + j] -= b[j];
39                 if (d[i + j] < 0) {
40                     d[i + j + 1] -= 1;
41                     d[i + j] += 10;
42                 }
43             }
44             // 使商的这一位增加 1
45             c[i] += 1;
46             // 返回循环开头，重新检查
47         }
48     }
49 }

```

入门篇完成！

将上面介绍的四则运算的实现结合，即可完成开头提到的计算器程序。

► calculator.cpp

压位高精度

引入

在一般的高精度加法，减法，乘法运算中，我们都是将参与运算的数拆分成一个个单独的数码进行运算。

例如计算 $\frac{1}{3}$ 时，如果按照高精度乘高精度的计算方式，我们实际上算的是 $\frac{1}{3}$ 。

在位数较多的时候，拆分出的数也很多，高精度运算的效率就会下降。

有没有办法作出一些优化呢？

注意到拆分数字的方式并不影响最终的结果，因此我们可以将若干个数码进行合并。

过程

还是以上面这个例子为例，如果我们每两位拆分一个数，我们可以拆分成 $\frac{1}{3}$ 。

这样的拆分不影响最终结果，但是因为拆分出的数字变少了，计算效率也就提升了。

从 [进位制](#) 的角度理解这一过程，我们通过在较大的进位制（上面每两位拆分一个数，可以认为是在 10^2 进制下进行运算）下进行运算，从而达到减少参与运算的数字的位数，提升运算效率的目的。

这就是 **压位高精度** 的思想。

下面我们给出压位高精度的加法代码，用于进一步阐述其实现方法：

► [压位高精度加法参考实现](#)

压位高精下的高效竖式除法

在使用压位高精时，如果试商时仍然使用上文介绍的方法，由于试商次数会很多，计算常数会非常大。例如在万进制下，平均每个位需要试商 5000 次，这个巨大的常数是不可接受的。因此我们需要一个更高效的试商办法。

我们可以把 double 作为媒介。假设被除数有 4 位，是 a ，除数有 3 位，是 b ，那么我们只要试一位的商：使用 10^2 进制，用式子 $\frac{a}{b}$ 来估商。而对于多个位的情况，就是一位的写法加个循环。由于除数使用 3 位的精度来参与估商，能保证估的商 q' 与实际商 q 的关系满足 $q' \leq q < 10q'$ ，这样每个位在最坏的情况下也只需要两次试商。但与此同时要求 double 的有效精度内，即 10^6 ，所以在运用这个方法时建议不要超过 10^6 进制，否则很容易因精度不足产生误差从而导致错误。

另外，由于估的商总是小于等于实际商，所以还有再进一步优化的空间。绝大多数情况下每个位只估商一次，这样在下一个位估商时，虽然得到的商有可能因为前一位的误差造成试商结果大于等于 base ，但这没有关系，只要在做统一进位便可。举个例子，假设 base 是 10，求 $\frac{1}{3}$ ，试商计算步骤如下：

1. 首先试商计算得到 $\frac{1}{3}$ ，于是 $\frac{1}{3}$ ，这一步出现了误差，但不用管，继续下一步计算。
2. 对余数 98801 继续试商计算得到 $\frac{1}{3}$ ，于是 $\frac{1}{3}$ ，这就是最终余数。
3. 把试商过程的结果加起来并处理进位，即 $\frac{1}{3}$ 便是准确的商。

方法虽然看着简单，但具体实现上很容易进坑，所以下面提供一个经过多番验证确认没有问题的实现供大家参考，要注意的细节也写在注释当中。

► [压位高精度高效竖式除法参考实现](#)

Karatsuba 乘法

记高精度数字的位数为 n ，那么高精度—高精度竖式乘法需要花费 $O(n^2)$ 的时间。本节介绍一个时间复杂度更为优秀的算法，由前苏联（俄罗斯）数学家 Anatoly Karatsuba 提出，是一种分治算法。

考虑两个十进制大整数 A 和 B ，均包含 n 个数码（可以有前导零）。任取 m ，记

其中 A_1, A_2, B_1, B_2 可得

观察知

于是计算 10^k ，只需计算 10^{k-1} ，再与 10^{k-2} 相减即可。

上式实际上是 Karatsuba 算法的核心，它将长度为 n 的乘法问题转化为了 $\frac{n}{2}$ 个长度更小的子问题。若令 $T(n)$ 记 Karatsuba 算法计算两个 n 位整数乘法的耗时为 $T(n)$ ，则有 $T(n) = 3T(\frac{n}{2}) + O(n)$ ，由主定理可得 $T(n) = O(n^{\log_2 3})$ 。

整个过程可以递归实现。为清晰起见，下面的代码通过 Karatsuba 算法实现了多项式乘法，最后再处理所有的进位问题。

► karatsuba_mulc.cpp

► 关于 new 和 delete

但是这样的实现存在一个问题：在 10 进制下，多项式的每一个系数都有可能达到 10^k 量级，在压位高精度实现中可能造成整数溢出；而若在多项式乘法的过程中处理进位问题，则 10^k 与 10^k 的结果可能达到 10^{2k} ，增加一个位（如果采用 10^9 的计算方式，则不得不特殊处理负数的情况）。因此，需要依照实际的应用场景来决定采用何种实现方式。

基于多项式的高效大整数乘法

如果数据规模达到了 10^6 或更大，普通的高精度乘法可能会超时。本节将介绍用多项式优化此类乘法的方法。

对于一个 n 位的十进制整数，可以将它看作一个每位系数均为整数且不超过 10^9 的多项式。这样，我们就将两个整数乘法转化为了两个多项式乘法。

普通的多项式乘法时间复杂度仍是 $O(n^2)$ ，但可以用多项式一节中的 [快速傅里叶变换](#)、[快速数论变换](#) 等算法优化，优化后的时间复杂度是 $O(n \log n)$ 。

封装类

[这里](#) 有一个封装好的高精度整数类，以及 [这里](#) 支持动态长度及四则运算的超迷你实现类。

► 这里是另一个模板

习题

- [NOIP 2012 国王游戏](#)
- [SPOJ - Fast Multiplication](#)
- [SPOJ - GCD2](#)
- [UVa - Division](#)
- [UVa - Fibonacci Freeze](#)
- [Codeforces - Notepad](#)

参考资料与链接

1. [Karatsuba algorithm - Wikipedia](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[Marcythm](#), [NachtgeistW](#), [StudyingFather](#), [383494](#), [AndrewWayne](#), [ayuusweetfish](#),
[Baobaobear](#), [CBW2007](#), [CCXXXI](#), [Chihaya-Yuka](#), [Chrogeek](#), [ChungZH](#), [countercurrent-time](#), [diauweb](#),
[Early0v0](#), [Enter-tainer](#), [gi-b716](#), [Glenn-Su](#), [H-J-Granger](#), [hsfzLZH1](#), [iamtwz](#), [Ir1d](#), [justarandomstring](#), [ksyx](#),
[Iss233](#), [ouuan](#), [shuzhouliu](#), [sshwy](#), [SukkaW](#), [Tiphereth-A](#), [TrisolarisHD](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用